



Cairo University

Faculty of Economics and Political Science

Socio-Computing Department

C201: Computer Applications in Social Sciences II

MODULE 3: PYTHON FOR DATA SCIENCE

LECTURE 3_1

INTRODUCTION TO DATA SCIENCE AND PANDAS

DR. AMAL SAKR

MODULE 3: LEARNING OBJECTIVES

By the end of this module, you will be able to understand and apply the fundamental concepts, tools, and techniques of data science to explore **simple** data-driven insights in a societal context. You will gain foundational knowledge in data manipulation and visualization using Python's relevant libraries.

The learning objective covers:

➤ Practical skills:

- Define data science and main concepts
- Studying [Pandas](#), which is a highly powerful and flexible Python library widely used in data science for data manipulation, analysis, and visualization. It provides simple and efficient tools to handle large datasets
- Perform exploratory analysis and summarize key features using descriptive statistics and visualization tools to uncover patterns or insights

OUTLINES

- Introduction to Data Science
- Fundamental Python Libraries for Data Scientists
- Create a Pandas DataFrame
- Getting a DataFrame Information
- Accessing the data from a DataFrame
- Updating a DataFrame
- DataFrame Methods
- Reading and Writing CSV File
- Visualization using pandas
- Case Study



What is Data Science?

Data Science (DS) is an interdisciplinary field combining methodologies from *statistics*, *computer science*, and *domain expertise*. Its objective is to systematically analyze both *structured* and *unstructured* data to **uncover patterns**, **extract insights**, and **solve real-world problems** through informed, data-driven decision-making.

Data Science Objectives

- **Data Preparation:** How to restructure, clean, and organize data for analysis.
- **Data Visualization & Communication:** Effectively presenting complex data findings and insights through visual tools and clear narratives.
- **Advanced Analytics:** Developing and applying *Machine Learning algorithms*—the backbone of Data Science—to *automatically learn and improve from data*, thereby *discovering hidden patterns, extracting insights, and building predictive models to forecast future outcomes*.
- **Ethical Considerations:** Understanding the ethical and social implications of collecting, analyzing, and using data.

What is Machine Learning?

Machine learning (**ML**) is a branch of artificial intelligence that enables computers to learn from data and make decisions or predictions without explicit programming.

In the first part of this course, you learned that there are many different algorithms just to sort a list of numbers.

Similarly, Machine Learning offers many different algorithms (**techniques**) to learn from data, such as *regression*, *decision trees*, and *neural networks*. Each algorithm is best suited for a specific type of problem.

Learning from data can mean different things depending on the goal.

For example, in the telecommunications industry, a company might use ML to understand why customers cancel their service (churn).

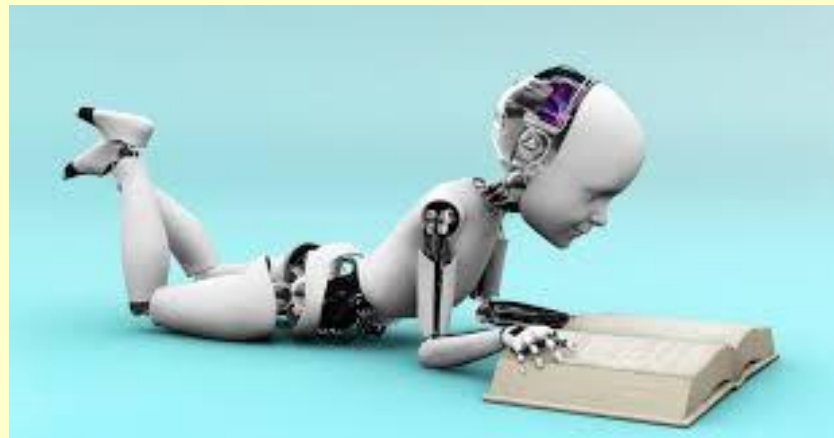
By predicting which customers are at risk of leaving, the company can offer them targeted discounts to encourage them to stay.

What is Machine Learning?

So, in general, **learning from data means** *using algorithms to identify hidden patterns within historical data.*

A **pattern** means reliable relationships between different variables, like "high bill" + "low usage" = "likely to leave".

Unlike following explicit, pre-programmed rules (like in sorting), the computer **discovers** and **uses these patterns** to build a predictive model that can generalize its knowledge to make accurate decisions about new, unseen information (the possibility of a new customer churns).



Artificial Intelligence: The Big Picture

Artificial Intelligence (AI) is a broad field of computer science focused on creating intelligent systems capable of simulating human intelligence to perform tasks like reasoning, learning, and problem-solving.

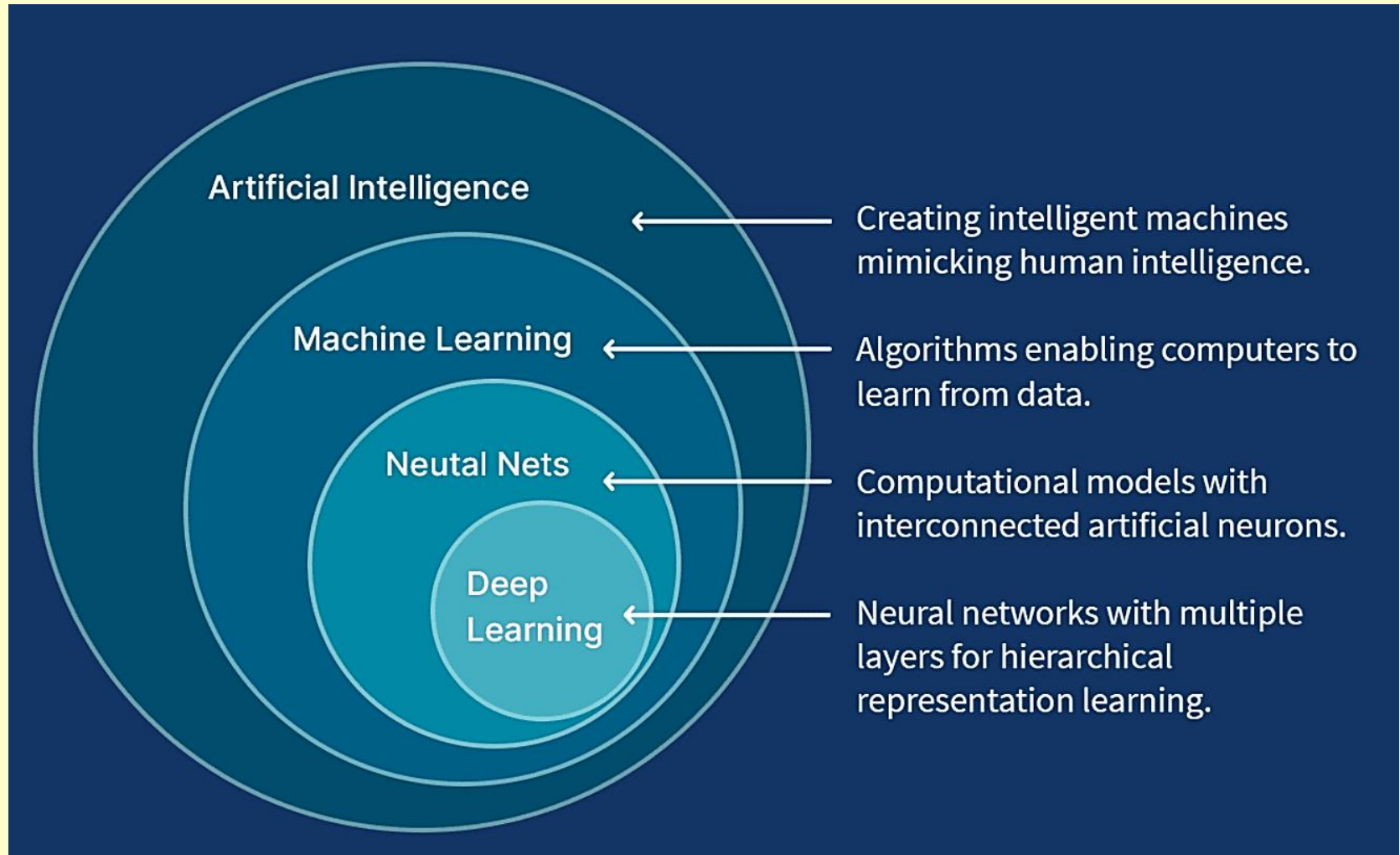
Think of Artificial Intelligence (AI) as the large umbrella for the ultimate ambition: creating systems capable of simulating human intelligence.

AI has many branches, encompassing anything that allows computers to mimic human intelligence, including **robotics**, Expert systems, and language recognition.

ML is a subset of **AI**, meaning that all machine learning is AI, but not all AI is machine learning. AI does not necessarily have to learn from data. For example, older Expert Systems make decisions based on a pre-written set of explicit IF/THEN rules.

ML is specifically focused on the development of algorithms that can learn from data. However, it is one of the most effective AI branches used today to achieve that ambition, specifically by learning from data instead of explicit rules.

Artificial Intelligence: The Big Picture



Data Science vs. Statistics

In Statistics, "**The Model is the King.**"

In Data Science, "**The Data is the King.**"

Statistics primarily focuses on **inference** and **understanding relationships** within data. It is largely **model-driven**, meaning it often begins by assuming a specific mathematical model (e.g., "Is X associated with Y?") and then tests if the data fits that assumption using methods like regression. The goal is to **explain phenomena, measure uncertainty, and draw reliable conclusions from structured data**. Its strength lies in the rigor of its methods and the interpretability of its findings, explaining why something is happening.

Data Science, conversely, is driven by **prediction, automation, and large-scale problem-solving**. It is **data-driven**, letting the data itself reveal patterns rather than starting with a predefined model. It combines statistical principles with advanced computational and Machine Learning algorithms to automatically learn from vast, diverse datasets (structured and unstructured). The goal is to **predict what will happen or how to make something happen**, ultimately to deliver actionable, data-driven solutions.

Test Your Knowledge

#	Question	Options	
1	Which three fields form the interdisciplinary foundation of Data Science (DS)?	A) Robotics, Expert Systems, and Statistics B) Statistics, Computer Science, and Domain Expertise C) Advanced Analytics, Data Preparation, and Ethics D) Python Programming, Visualization, and Modeling	B
2	What is the primary focus of traditional Statistics compared to Data Science?	A) Prediction and automation B) Building large neural networks C) Inference and explaining relationships within data D) Analyzing only unstructured data	C
3	Which statement correctly describes the relationship between Artificial Intelligence (AI) and Machine Learning (ML)?	A) ML is a subset of AI B) AI is a subset of ML C) AI and ML are two different names for the same concept D) ML is an older field than AI	A
4	What does the term "pattern" in the context of ML primarily refer to?	A) A single random data point or outlier B) A simple coin flip probability calculation C) The explicit rule written by the programmer D) A reliable statistical relationship between different variables	D
5	Which Data Science Objective is concerned with applying ML algorithms to discover patterns and forecast outcomes?	A) Data Preparation B) Ethical Considerations C) Data Visualization D) Advanced Analytics	D
6	Data Science is described as being primarily data-driven , which means it relies on:	A) Pre-defining a specific mathematical model before analysis B) Letting the data itself reveal patterns rather than starting with a predefined model C) Focusing only on cleaning and organizing data D) Only using model-driven methods like traditional regression	B

Fundamental Python Libraries for Data Scientists

The Python community is one of the most active programming communities with a huge number of developed toolboxes. The most popular Python toolboxes for any data scientist are *NumPy*, *SciPy*, *Pandas*, *Matplotlib*, *Seaborn* and *Scikit-Learn*.

- 1) **NumPy** is the cornerstone toolbox for scientific computing with Python. It provides, among other things, support for multidimensional arrays with basic operations on them and useful linear algebra functions. Many toolboxes use the **NumPy** array representations as an efficient basic data structure.
- 2) **Pandas** provides high-performance data structures and data analysis tools. it's like a programmatic excel sheet.
- 3) **Matplotlib** is a plotting library that has many tools for data visualization.
- 4) **Scikit-learn** is a **machine learning library** that offers simple and efficient tools for common tasks in data analysis such as classification, regression, clustering, dimensionality reduction, model selection, and preprocessing.

There are tens of others you can use that would fit your use cases, but 95% of all analyses can be run from start to finish with these four.

Pandas

- *Pandas* is an open-source, high-performance, easy-to-use data structure and data analysis tool for analyzing tabular data.
- Pandas was created by *Wes McKinney* in 2008 for financial data analysis at AQR Capital Management, a major US investment firm. The name is derived from the term **"Panel Data."**
- The key feature of Pandas is its ability to provide a fast, flexible, and efficient object called a **DataFrame** that makes data manipulation, analysis, and transformation easy and intuitive.
- *Pandas* also has tools for importing and exporting data from different formats: comma-separated value (CSV), text files, Microsoft *Excel*, and *SQL* databases. Furthermore, Pandas provides a convenient *Matplotlib* interface.
- We can import pandas as follows: **import pandas as pd**

It is a standard convention in the documentation to use *pd* as an alias; however, you do not have to use that convention.

What is a DataFrame?

A **DataFrame** is a 2-dimensional data structure, like a 2-dimensional array or a table with rows and columns. The **DataFrame** structure can be seen as a spreadsheet that offers very flexible ways of working with it.

Ex: consider the table below

	Math	Science
Ali	90	88
Mona	85	92
Sara	78	80

In Python, this table is represented by a DataFrame, a data type composed of:

Rows → individual records represent the information of three students (here named: Ali, Mona, Sara)

Columns → variables/features (here: Math, Science)

Cells → actual data values (e.g., Ali scored 90 in Math)

A DataFrame can be seen as *a dictionary of columns* with enhanced capabilities, including indexing, slicing, alignment, aggregation, plotting, merging, handling missing data, and many other built-in methods that a standard dictionary does not provide.

DataFrame

Attributes and Methods

The DataFrame class is powerful because it comes with a suite of built-in features that simplify common data analysis tasks. These features are known as attributes and methods.

An **attribute** is a value associated with an object or class that is referenced by name using dotted expressions.

A **method** is a function that is defined inside a class body and typically performs an action.

A simpler way of thinking about the distinction between attributes and methods is to remember that attributes are characteristics of the object, while methods are actions or operations.

DataFrame: Attributes

(You don't add parentheses to use it)

Attribute	Description
<u>columns</u>	Returns the column labels of the DataFrame
<u>dtypes</u>	Returns the data types in the dataframe
<u>iloc</u>	Accesses a group of rows and columns using integer-based indexing
<u>loc</u>	Accesses a group of rows and columns by label(s) or a Boolean array
<u>shape</u>	Returns a tuple representing the dimensionality of the DataFrame
<u>index</u>	Returns all the row labels of a DataFrame
<u>values</u>	Returns the underlying data of a DataFrame

There are many, many more. For a more detailed list, refer to the [pandas DataFrame documentation](#), which includes helpful examples of how to use each tool.

DataFrame: *Common Methods*

Method	Description
<u>apply()</u>	Applies a function over an axis of the dataframe
<u>copy()</u>	Makes a copy of the dataframe's indices and data
<u>describe()</u>	Returns descriptive statistics of the dataframe, including the minimum, maximum, mean, and percentile values of its numeric columns; the row count; and the data types
<u>drop()</u>	Drops specified labels from rows or columns
<u>to_datetime</u>	Convert the type of the variable into datetime type.
<u>groupby()</u>	Splits the dataframe, applies a function, and combines the results
<u>head(n=5)</u>	Returns the first <i>n</i> rows of the dataframe (default=5)
<u>info()</u>	Returns a concise summary of the dataframe
<u>isna()</u> / <u>isnull()</u>	Returns a same-sized Boolean dataframe indicating whether each value is null (can also use <i>isnull()</i> as an alias)
<u>sort_values()</u>	Sorts by the values across a given axis
<u>value_counts()</u>	Returns a series containing counts of unique rows in the dataframe
<u>where()</u>	Replaces values in the dataframe where a given condition is false

For a complete list of pandas' methods, refer to

<https://pandas.pydata.org/docs/reference/frame.html>

DataFrame Manipulation

In the next few slides, we will learn

- How to *create a DataFrame*
- How to *access its data and its attributes*
- How to *update a DataFrame*:
 - Adding a column to the DataFrame,
 - Removing an existing column from a DataFrame,
 - Modifying an existing column in a DataFrame,
 - Changing the data type of column values in a DataFrame
- How to implement *DataFrame methods*

Three Methods to Create the Same DataFrame.

#----- 1. Creating DataFrames using List of Lists -----

```
import pandas as pd
f = ['Names', 'Country', 'City'] #Fields Names
d = [['Mohammed', 'Egypt', 'Banha'], # Fields values ... row by row
      ['Ann', 'UK', 'London'],
      ['John', 'Sweden', 'Stockholm']]
df = pd.DataFrame(columns = f, data = d) ; print(df)
```

#----- 2. Creating DataFrames using a Dictionary-----

```
data = {'Name': ['Mohammed', 'David', 'John'], 'Country': ['Egypt',
'UK', 'Sweden'], 'City': ['Banha', 'London', 'Stockholm']}
#Set of Fields Names and Values ... col by col
df = pd.DataFrame(data); print(df)
```

#----- 3. Creating DataFrames using list of Dictionaries-----

```
data = [{'Name': 'Mohammed', 'Country': 'Egypt', 'City': 'Banha'},
        {'Name': 'David', 'Country': 'UK', 'City': 'London'},
        {'Name': 'John', 'Country': 'Sweden', 'City': 'Stockholm'}]
#List of Sets of Fields Names and Values ... row by row
df = pd.DataFrame(data); print(df)
```

print(df)

Column Names

Column Names			
	Names	Country	City
0	Mohammed	Egypt	Banha
1	Ann	UK	London
2	John	Sweden	Stockholm

index

Values

Getting a DataFrame Attributes

```
#----- Access different information about the DataFrame -----  
import pandas as pd  
  
f = ['Names', 'Country', 'City'] #Fields Names  
d = [['Mohammed', 'Egypt', 'Banha'], # Fields values  
     ['Ann', 'UK', 'London'],  
     ['John', 'Sweden', 'Stockholm']]  
  
df = pd.DataFrame(columns = f, data = d) ; print(df)  
  
print(df) #print cols names and values  
print(df.shape) #Out: (3, 3) → (no. of rows, no. of columns)  
print(df.size) #Out: 9 → no. of rows * no. of columns  
print(df.columns) # Out: Col Names ['Names', 'Country', 'City']  
print(df.index) #print index values  
print (df.dtypes) # get the type of every column  
print(df.info()) #print all info about columns
```

Output

`print(df.index)`

RangeIndex: 3 entries, (start=0, stop=3, step=1)

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Names	3 non-null	object
1	Country	3 non-null	object
2	City	3 non-null	object

`df.info()`

dtypes: object(3)

`print(df.dtypes)`

memory usage: 200.0+ bytes

`df.columns`

Names **Country** **City**

0	Mohammed	Egypt	Banha
1	Ann	UK	London
2	John	Sweden	Stockholm

Values

`print(df)`

index

Data Types

Int64: refers to integer64. It means that the datatype contains integers between -2^{63} to 2^{63} . i.e., integer somewhere between negative 9 quintillion and positive 9 quintillion (quintillion = 10^{18}).

Object: In pandas, usually means string data, although it can technically store any Python object like lists, custom objects, etc.

Accessing the Data from a DataFrame

```
import pandas as pd
f = ['Names', 'Country', 'City'] #Fields Names
d = [['Mohammed', 'Egypt', 'Banha'], # Fields values
      ['Ann', 'UK', 'London'],
      ['John', 'Sweden', 'Stockholm']]

df = pd.DataFrame(columns = f, data = d); print(df)

print(df.head()) # get first 5 rows
print(df.head(20)) # get first 20 rows ; you can select any No.
print(df.tail()) # get last 5 rows
print(df.tail(10)) # get last 10 rows
print(df.sample(2)) # get 3 random rows

#Access Columns

print(df['Names']) #print the values of the 'Names' col only

print(df.Names) #It's possible to select columns using dot
notation instead of bracket notation; bracket notation is often
preferred.

df[['Names', 'City']] # print the values of the 'Names' & 'City'
```

Accessing the Data from a DataFrame

```
import pandas as pd
f = ['Names', 'Country', 'City'] #Fields Names
d = [['Mohammed', 'Egypt', 'Banha'], # Fields values
      ['Ann', 'UK', 'London'],
      ['John', 'Sweden', 'Stockholm']]

df = pd.DataFrame(columns = f, data = d); print(df)

#Access Rows using iloc
print(df.iloc[0:2]) #print certain rows; here from rows 0 to 1
print(df.iloc[[0, 2]]) #print certain rows; here rows 0 and 2
print(df.values) #list of lists(rows values)
print(df.values[1]) #Out: ['Ann' 'UK' 'London']

#Access rows and columns
print(df.iloc[:, [0,2]])
# iloc access all rows for specific column indices
print(df.loc[:, ['Names', 'City']])
# loc access all rows for specific column names
# Both commands get ['Names', 'City'] for all rows

print(df["City"][2]) # Out: Stockholm → print certain cell
print(df["Country"][0:2]) #Out: Egypt, UK
```

Output

			<code>df["Country"][0:2]</code>
			<code>df.iloc[0:2]</code> <code>= df.head(2)</code>
	Names	Country	City
0	Mohammed	Egypt	Banha
1	Ann	UK	London
2	John	Sweden	Stockholm

`df['Names']`

`df["City"][2]`

Examples for slicing using `iloc`

<code>df.iloc[5:10, 2:]</code>	→ selects only rows 5 through 9, from columns 2 to last one
<code>df.iloc[5:10]</code>	→ selects only rows 5 through 9, all columns
<code>df.iloc[1, 2]</code>	→ selects value at row 1, column 2
<code>df.iloc[[0, 2], [2, 4]]</code>	→ selects only rows 0 and 2, at columns 2 and 4

Example: Calculate the BMI Value

```
import pandas as pd

def Fill_BMI_Status (b):
    status = []
    for b in b:
        if (b <= 18.5): s= "Under Weight"
        if (b > 18.5 and b < 25): s= "Healthy Weight"
        if (b >= 25 and b <= 30): s= "Over Weight"
        if (b > 30): s= "Obese"
        status.append(s)
    return status

#----- Creating a DataFrame and modifying it -----
df = pd.DataFrame(columns=['Name','Weight', 'Height'])# Empty df
df['Name'] = ['Ahmed','Ngwa','Salwa']
df['Weight'] = [55, 78, 100]
df['Height'] = [1.73, 1.60, 1.65]
df['BMI'] = round(df['Weight']/df['Height']**2 ,2)

#- Calling calculate_bmi Function and adding two new cols to df --
df['Status'] = Fill_BMI_Status(df['BMI'])
df = df.sort_values(by='BMI', ascending=False); print(df)
```

Example: Calculate the BMI Value

# Output					
	Name	Weight	Height	BMI	Status
2	Salwa	100	1.65	36.73	Obese
1	Ngwa	78	1.60	30.47	Obese
0	Ahmed	55	1.73	18.38	Under Weight

Updating a DataFrame

#----- Change index and column name -----

```
import pandas as pd

f = ['Names', 'Country', 'City']    # Column Names
d = [['Mohammed', 'Egypt', 'Banha'], ['Ann', 'UK', 'London'],
      ['John', 'Sweden', 'Stockholm']]    #row by row values

df = pd.DataFrame(columns = f, data = d) ; print(df)
df.index = ["a", "b", "c"]; print(df)    #change the index
print(df["Country"]["a"])

    #Out: Egypt; access cells using new index

#Change the name of the first col from "Name" to "Customer_Name"
df.columns = ["Customer_Name", "Country", "City"] ; print(df)
```

Adding Rows to a DataFrame

#----- Add a new row using two methods -----

```
import pandas as pd

f = ['Names', 'Country', 'City']    #Fields Names ... row by row
d = [['Mohammed', 'Egypt', 'Banha'], ['Ann', 'UK', 'London'],
      ['John', 'Sweden', 'Stockholm']]    #rows values

df = pd.DataFrame(columns = f, data = d) ; print(df)

new_row = ["Kate", "France", "Paris"]

# Method 1: Using the concat method

df2 = pd.DataFrame(data= [new_row], columns=f) #create a new df
print("New DataFrame\n",df2)

df = pd.concat([df, df2], ignore_index = True) # join two dfs
print("DataFrame after Concatenation\n",df)

# Method 2: Using the loc method

new_row = ["Kate", "France", "Paris"]

n = len(df); df.loc[n] = new_row

print("DataFrame after setting the new location\n",df)
```

Output

New DataFrame

	Names	Country	City
0	Kate	France	Paris

DataFrame after Concatenation

	Names	Country	City
0	Mohammed	Egypt	Banha
1	Ann	UK	London
2	John	Sweden	Stockholm
3	Kate	France	Paris

DataFrame after setting the new location

	Names	Country	City
1	Ann	UK	London
2	John	Sweden	Stockholm
3	Kate	France	Paris
4	Kate	France	Paris
5	Kate	France	Paris

Adding Columns to a DataFrame

```
import pandas as pd

f = ['Names', 'Country', 'City']    #Columns Names
d = [['Mohammed', 'Egypt', 'Banha'], ['Ann', 'UK', 'London'],
      ['John', 'Sweden', 'Stockholm']]    #row by row values

df = pd.DataFrame(columns = f, data = d) ; print(df)

""" Method 1: Add a new col called "Gender" at the end and
set its values =1 for all records """

df["Gender"] =1; print(df)    # a new col

""" Method 2: Add a new col in position 1 and set its values
= 2 for all records """
df.insert(1, "Marital_Status", 2) ; print(df);
```

df

	Customer_Name	Marital_Status	Country	City	Gender
a	Mohammed	2	Egypt	Banha	1
b	Ann	2	UK	London	1
c	John	2	Sweden	Stockholm	1

Deleting Columns from a DataFrame

Method 1: using the del built-in function

```
del df["Marital_Status"]
```

```
print("DataFrame after del Marital_Status\n",df)
```

Method 2: using the pop method

```
x = df.pop("Gender")
```

```
print("DataFrame after pop Gender\n",df)
```

Method 3: using the drop method

```
x = df.drop(columns="Customer_Name") ;
```

```
print("Returned DataFrame after dropping Customer_Name\n", x)
```

```
print("DataFrame after dropping \n",df) # df does not change
```

Two solutions to get the changes after the drop method

```
df = x ; print(df) #First, assign df to the output x
```

```
df = df.drop(columns="Country", inplace = True)
```

#Second: set inplace = True

Deleting Rows from a DataFrame

```
x = df.drop(["a", "c"]); # delete 2 Rows "a" and "c"
print("Returned df\n", x)
print("DataFrame after dropping two rows\n", df)
df.drop(["a", "c"], inplace = True) # Save the changes in df
print("DataFrame after dropping two rows,
inplace=True\n", df)
```

DataFrame Methods: *Summary Statistics*

```
import pandas as pd

df = pd.DataFrame({"GDP": [119, 206, 240, 94],
                  "median_age": [42, 47, 52, 45],
                  "population": [65, 60, 83, 46],
                  "pop_gr": [0.20, -0.15, 0.20, 0.04]},
                 index = ['France', 'Italy', 'Germany', 'Spain'])

print(df.count()) #No. of Rows

df.sum();         #The sum of each column

df.cumsum         #The cumulative sum of each column

print(df.mean(axis=0))
                # get the mean of columns; axis=1 for rows

print(df["GDP"].mean())
                # get the mean of a specific column; GDP
```

DataFrame Methods: *Summary Statistics*

```
vstat = df.agg(['mean', 'median', 'std'])  
  
print(vstat) # the 3 statistics for the 4 numeric cols  
  
print(vstat.columns, "\n", vstat.index)  
  
print(vstat['GDP']) # the 3 statistics for the GDP only  
  
print(df.std(axis=1))  
  
#the std of all rows; which is nonsense
```

Note: agg (stands for aggregate) method is used to apply multiple functions to a dataframe at the same time. Note that vstat is a dataframe

Summary Statistics using

df.describe()

For **numerical** column, the `describe()` method gets **8 descriptive statistics**:

count : Number of non-null observations	25% : The first quartile (25th percentile)
mean : The arithmetic average	50% : The median (50th percentile)
std : The standard deviation	75% : The third quartile (75th percentile)
min : The smallest (minimum) value	max : The largest (maximum) value

For a **categorical** column, the `describe()` method gets **4 descriptive statistics**:

count : Number of non-NA/null observations
unique : Number of unique values
top : The most common value (the mode)
freq : The frequency of the most common value

Note: If a dataframe consists of a mix of numerical and categorical variables, `df.describe()` will get the 8 statistics for numerical variables only.

DataFrame Methods: *Summary Statistics*

```
import pandas as pd
df = pd.DataFrame({"GDP": [119, 206, 240, 94],
                  "median_age": [42, 47, 52, 45],
                  "population": [65, 60, 83, 46],
                  "pop_gr": [0.20, -0.15, 0.20, 0.04]},
                 index = ['France', 'Italy', 'Germany', 'Spain'])

df.describe()
    #gets the 8 descriptive statistics for the 4 cols
print(df['pop_gr'].unique()) #out: [ 0.2  -0.15  0.04]
print(round(df.corr(), 2)) #generate correlation matrix
print(round(df.corr()['GDP'], 2))
    #correlation of density with all other vars
print(df.corr()['GDP']['population'])
    #corr between density and population = 0.78
```

DataFrame Methods: *Summary Statistics*

Output of `df.describe()`

	GDP	median_age	population	pop_gr
count	4.00	4.00	4.00	4.00
mean	164.75	46.50	63.50	0.07
std	69.43	4.20	15.29	0.17
min	94.00	42.00	46.00	-0.15
25%	112.75	44.25	56.50	-0.01
50%	162.50	46.00	62.50	0.12
75%	214.50	48.25	69.50	0.20
max	240.00	52.00	83.00	0.20

Output of `round(df.corr(),2)`

	GDP	median_age	population	pop_gr
GDP	1.00	0.85	0.78	-0.09
median_age	0.85	1.00	0.65	0.03
population	0.78	0.65	1.00	0.53
pop_gr	-0.09	0.03	0.53	1.00

Output of `round(df.corr()['GDP'],2)`

GDP	1.00
median_age	0.85
population	0.78
pop_gr	-0.09

DataFrame Methods: *Missing Data*

```
data=pd.DataFrame({"A":[1, 2.1, None, 4.7, 5.6, 6.8],
                  "B":[.25, None, None, 4, 12.2, 14.4]});

print(data)

print(data.isna())

print(data.isna().sum()) #get the sum of null values for each col

data.dropna() #print the results of removing rows contain null val

print(data) #the data dataframe has not been changed

data_1 = data

data_1.dropna(subset=['A'], inplace=True)

print(data_1) #data_1 has changed

data_2=data.dropna(axis = 1); print(data_2) #del cols contain null
```

data

	A	B
0	1.0	0.25
1	2.1	NaN
2	NaN	NaN
3	4.7	4.00
4	5.6	12.20
5	6.8	14.40

print(data.
isna())

	A	B
0	False	False
1	False	True
2	True	True
3	False	False
4	False	False
5	False	False

print(data.
isna().sum())

A	1
B	2

print(data_1)

	A	B
0	1.0	0.25
1	2.1	NaN
3	4.7	4.00
4	5.6	12.20
5	6.8	14.40

print
(data_2)

	A
0	1.0
1	2.1
3	4.7
4	5.6
5	6.8

DataFrame Methods: **groupby()**

The `groupby()` method is used to split a DataFrame into groups based on the values of one or more columns, then apply an aggregation or transformation function to each group.

It follows the classic "split–apply–combine" strategy:

- Split the data into groups
- Apply a function (e.g., `mean()`, `sum()`, `count()/size()`, `min()`, `max()`). Multiple functions can be applied using `agg()` method.
- The result of the `groupby()` method is a new DataFrame or Series. The index of the result is the variable that is used after the parameter `by`.

For example, `res= df.groupby("Year")`. Then `res` is a series whose index is "Year"

Why Use `groupby()` ?

- data by categories (e.g., average salary by department)
- Apply different calculations to subgroups
- Analyze patterns across categorical values

DataFrame Methods: **groupby()**

```
import pandas as pd
df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],
                  'data': range(6)}); print(df)

x = df.groupby('key'); dir(x) #get all methods and attributes of x
x.size() # No. of rows in each group
print ("Sum of each group\n", x.sum())
print ("Statistics Description for each group \n", x.describe())
df.groupby('key').get_group('A').describe() # specific group
```

	key	data
0	A	0
1	B	1
2	C	2
3	A	3
4	B	4
5	C	5

x.size()

key	
A	2
B	2
C	2

Sum of each group

key	
A	3
B	5
C	7

Statistics Description for each group

	count	mean	std	min	25%	50%	75%	max
key								
A	2.0	1.5	2.12132	0.0	0.75	1.5	2.25	3.0
B	2.0	2.5	2.12132	1.0	1.75	2.5	3.25	4.0
C	2.0	3.5	2.12132	2.0	2.75	3.5	4.25	5.0

.get_group('A')
.describe()

Example

Department	Employee	Salary	HoursW orked	Year
HR	Alice	50000	160	2023
HR	Bob	52000	155	2023
HR	Carol	51000	162	2024
HR	Dan	53000	158	2024
IT	Eve	60000	170	2023
IT	Frank	61000	165	2023
IT	Grace	61000	168	2024
IT	Heidi	63000	172	2024
Finance	Ivan	70000	150	2023
Finance	Judy	71000	155	2023
Finance	Karl	70000	160	2024
Finance	Laura	72000	165	2024

Use `groupby()` method to compute

1. The average salary per department.
2. The average salary per department per year.
3. The mean and standard deviation of salaries by department and year.
4. The average salary and average hours worked by department and year.
5. The number of employees per department per year.

Example

```
import pandas as pd

data = {
    'Department': ['HR', 'HR', 'HR', 'HR',
                   'IT', 'IT', 'IT', 'IT',
                   'Finance', 'Finance', 'Finance', 'Finance'],
    'Employee': ['Alice', 'Bob', 'Carol', 'Dan',
                 'Eve', 'Frank', 'Grace', 'Heidi',
                 'Ivan', 'Judy', 'Karl', 'Laura'],
    'Salary': [50000, 52000, 51000, 53000,
               60000, 61000, 61000, 63000,
               70000, 71000, 70000, 72000],
    'HoursWorked': [160, 155, 162, 158,
                    170, 165, 168, 172,
                    150, 155, 160, 165],
    'Year': [2023, 2023, 2024, 2024,
             2023, 2023, 2024, 2024,
             2023, 2023, 2024, 2024] }

df = pd.DataFrame(data)
```

Example

```
print("1. Average salary per department:\n")
print(df.groupby('Department')['Salary'].mean())

print("2. Average salary per department per year:\n")
print(df.groupby(['Department',
'Year'])['Salary'].mean().unstack())
    #unstack() turns a row index into a column; useful for
    multiple index levels results

print("3. Mean and std of salaries by department and
year:\n")
print(df.groupby(['Department',
'Year'])['Salary'].agg(['mean', 'std']).unstack())

print("4. Average of salary and hours_worked by department
and year:\n")
g = df.groupby(['Department', 'Year'])[['Salary',
'HoursWorked']].agg(['mean', 'std']).unstack()
print(g)

print("5. Number of employees per department per year:\n")
print(df.groupby(['Department', 'Year']).size().unstack())
```

Output

1. Average salary per department:

```
df.groupby('Department')['Salary'].mean()
```

Department

Finance	70750.0
HR	51500.0
IT	61250.0

2. Average salary per department per year:

```
df.groupby(['Department', 'Year'])['Salary'].mean().unstack()
```

Year	2023	2024
------	------	------

Department

Finance	70500.0	71000.0
HR	51000.0	52000.0
IT	60500.0	62000.0

3. Mean and std of salaries by department and year:

```
df.groupby(['Department', 'Year'])['Salary'].agg(['mean', 'std']).unstack()
```

	mean		std	
Year	2023	2024	2023	2024

Department

Finance	70500.0	71000.0	707.106781	1414.213562
HR	51000.0	52000.0	1414.213562	1414.213562
IT	60500.0	62000.0	707.106781	1414.213562

Output

4. Average of salary and hours_worked by department and year:

```
df.groupby(['Department', 'Year'])[['Salary',  
'HoursWorked']].agg(['mean', 'std']).unstack()
```

	Salary				... HoursWorked		
	mean		std		...		
Year	2023	2024	2023	...	2024	2023	
2024							
Department				...			
Finance	70500.0	71000.0	707.106781	...	162.5	3.535534	3.535534
HR	51000.0	52000.0	1414.213562	...	160.0	3.535534	2.828427
IT	60500.0	62000.0	707.106781	...	170.0	3.535534	2.828427

5. Number of employees per department per year:

```
df.groupby(['Department', 'Year']).size().unstack()
```

Year	2023	2024
Department		
Finance	2	2
HR	2	2
IT	2	2

DataFrame Methods: *Method Chaining*

- In pandas, we can chain multiple DataFrame methods together to perform complex data transformations or analyses concisely and readably.
- This approach is called **method chaining**, and it allows us to apply several operations sequentially without the need for intermediate variables.
- The **agg()** method in pandas allows you to apply one or more aggregation functions (like sum, mean, count, min, max, etc.) to a DataFrame. It is often used after groupby() to calculate summary statistics for different groups in the data.

```
import pandas as pd
df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'], 'data': range(6)}); print(df)
print(df.groupby('key').sum())
print(df.groupby('key').agg(['sum', 'count']))
print(df.groupby('key')['data'].agg(['sum', 'count']))
# Equivalent code but allows selecting a specific col
print(df.groupby('key').describe())
```

.agg(['sum', 'count'])		
	sum	count
key		
A	3	2
B	5	2
C	7	2

DataFrame Methods: *Filtering*

- Filtering is the process of selecting a subset of data **based on certain criteria or conditions**.
- Filtering data in pandas can be done using Boolean indexing or the `.query()` method.
 - **Boolean indexing** involves writing logical conditions directly within square brackets, such as `df[df['Country'] != 'UK']`, to filter rows based on column values.
 - In contrast, the **`.query()` method** provides a more readable, SQL-like syntax: `df.query("Country != 'UK')"`.
- Both approaches are powerful and return the same result. However, `.query()` *is often preferred* for cleaner syntax in more complex conditions, especially when working with string column names or combining multiple filters.

DataFrame Methods: *Filtering*

```
import pandas as pd
f = ['Names', 'Country', 'City']
d = [['Mohammed', 'Egypt', 'Banha'],
      ['Ann', 'UK', 'London'],
      ['John', 'Sweden', 'Stockholm']]
df = pd.DataFrame(d, columns=f) # Create the DataFrame

# ***** Example 1: Country == 'UK' *****
filter1 = df[df['Country'] == 'UK'] # df[True/False] column
print("Boolean Indexing: Country == 'UK'"); print(filter1, "\n")

query1 = df.query("Country == 'UK' ")
print("Query: Country == 'UK'"); print(query1, "\n")

# ***** Example 2: City != 'London' *****
condition = df['City'] != 'London' ; print(condition)
#return True/False col
filter2 = df[condition]
print("Boolean Indexing: City != 'London'"); print(filter2, "\n")

query2 = df.query("City != 'London'")
print("Query: City != 'London'"); print(query2, "\n")
```

Pandas: *Reading CSV File*

```
import pandas as pd

cur_file="D:\\Course_Second Year\\Datasets\\weight-height.csv"

df = pd.read_csv(cur_file) #comma-separated value

print(df.shape)           # get the data dimension (10000, 3)
print(df.info())          # get the data info
print(df.head())          # read the first 5 rows by default
print(df.head(15))        # read the first 15 rows
print(df.tail())          # read the last 5 rows by default
print(df.sample(20))      # get 20 random rows as a sample
```

	Gender	Weight	Height
0	Male	73.847017	180.893563
1	Male	68.781904	162.310473
2	Male	74.110105	212.740856
3	Male	71.730978	220.042470
4	Male	69.881796	206.349801
...			...
...			...
9995	Female	66.172652	136.777454
9996	Female	67.067155	170.867906
9997	Female	63.867992	128.475319
9998	Female	69.034243	163.852461
9999	Female	61.944246	113.649103

[10000 rows x 3 columns]

```
df.info()

RangeIndex: 10000 entries, 0 to 9999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Gender      10000 non-null  object
1   Height      10000 non-null  float64
2   Weight      10000 non-null  float64
dtypes: float64(2), object(1)
```

Pandas: *Reading CSV File*

```
#----- Reading CSV File -----  
print(df.columns) #Out: Index(['Gender', 'Weight', 'Height'])  
h = df['Height'] #get a specific column  
w = df['Weight']  
print( "Height values\n", round(h,2))  
print("Height Descriptive Statistics\n", round(h.describe(),2))  
print("Weight Descriptive Statistics\n", round( w.describe(),2))
```

Height values

0	180.89
1	162.31
2	212.74
3	220.04
4	206.35
...	...
9995	136.78
9996	170.87
9997	128.48
9998	163.85
9999	113.65

Height Descriptive Stat Info

count	10000.00
mean	161.43
std	32.10
min	64.70
25%	135.82
50%	161.21
75%	187.15
max	269.99
Name: Height, dtype: float64	

Weight Descriptive Stat Info

count	10000.00
mean	66.37
std	3.85
min	54.26
25%	63.51
50%	66.32
75%	69.17
max	79.00
Name: Weight, dtype: float64	

Pandas: *Reading CSV File*

```
print("Descriptive Statistics for Numerical variables")
print(df.describe())
df["Gender"].value_counts()
df.groupby("Gender").mean()
x= df.groupby("Gender").agg(["mean", "max", "min"])
print(round(x,2))
```

value_counts() returns a Series showing each unique value in a column and how many times it appears. It is frequently applicable to categorical columns.

Descriptive Statistics for numerical variables

	Weight	Height
count	10000.00	10000.00
mean	66.37	161.43
std	3.85	32.10
min	54.26	64.70
25%	63.51	135.82
50%	66.32	161.21
75%	69.17	187.15
max	79.00	269.99

value_counts()

Gender	
Male	5000
Female	5000

Mean(Weight) Mean(Height)

Gender		
Female	63.71	135.86
Male	69.03	187.01

Weight

Height

	mean	max	min	mean	max	min
Gender						
Female	63.71	73.39	54.26	135.86	202.24	64.7
Male	69.03	79.00	58.41	187.01	269.99	112.9

Pandas: *Write a dataframe to a CSV File*

```
import os, pandas as pd

cur_file="D:\\Course_Second Year\\Datasets\\weight-height.csv"

df = pd.read_csv(cur_file)          # read comma-separated value file
print(os.path.exists(cur_file))    #print True if the file exists

cur_dir = os.getcwd()
        #to get the current directory (path + current folder)
print("Current Directory: ", cur_dir) #current folder

X = round(df.describe(), 2)

X.to_csv("d:\\wh_DescCSV.csv") #save as CSV file on D:

X.to_excel("d:\\wh_DescXl.xlsx") #save as Excel file on D:
```

wh_Description.csv

```
,Weight, Height
count,10000.0,10000.0
mean,66.37,161.43
std,3.85,32.1
min,54.26,64.7
25%,63.51,135.82
50%,66.32,161.21
75%,69.17,187.15
max,79.0,269.99
```

wh_DescXl.xlsx

	Weight	Height
count	10000	10000
mean	66.37	161.43
std	3.85	32.1
min	54.26	64.7
25%	63.51	135.82
50%	66.32	161.21
75%	69.17	187.15
max	79	269.99

Sheet 5: Question 2

What are the outputs of the given below Python code?

```
#----- Create the dataframe-----  
  
import pandas as pd  
data = {  
    'Name': ['Ahmed', 'Mona', 'Youssef', 'Sara', 'Ahmed'],  
    'Department': ['Math', 'Physics', 'Math', 'Physics', 'Math'],  
    'Math Score': [78, 90, 88, 85, None],  
    'English Score': [80, 92, 91, None, 85],  
    "Addresses": [  
        '123 Cairo St, Cairo',  
        '456 Alexandria Rd, Alexandria',  
        '789 Luxor Ave, Giza', None,  
        '101 Tahrir Sq, Aswan'    ]  
}  
  
df = pd.DataFrame(data)  
print(df)
```

Sheet 5: Question 2

```
df.info()
df.shape
df.size
df.isna().sum()
df["GPA"] = 3 ; print(df)
df['Name'].unique()
df.iloc[2]['Addresses']
df['Addresses'].iloc[2].split()
df.iloc[:: -1].head(1)['Math Score']
df[df['Name'] == 'Ahmed']['Math Score']
df[df['English Score'] > 85]['Name']
df.query("Department == 'Math' and `Math Score` > 80")
df.groupby('Department').describe()
df.groupby('Department')['Math Score'].agg(["mean", "median"])
df.dropna(subset=['Addresses']); print(df)
df.dropna(subset=['Addresses'], inplace=True); print(df)
x= df[['Math Score', 'English Score']]; print(x.mean().max())
x= df['Department'] == 'Math'; print(df[x]['Name'].count())
df.sort_values(by='Math Score', ascending=False).head(3)
```

Python Visualization Libraries

Python gives us a massive toolbox for turning numbers into pictures. Depending on your goal, you have a few powerful options:

Matplotlib: Think of this as the foundation. It's a low-level library that gives you total control over every pixel (axes, grid lines, colors). It can make almost anything—static or animated—but it can be a bit verbose to write.

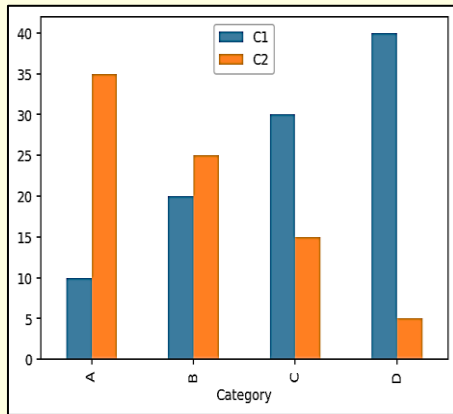
Seaborn: This is the "beautifier" built on top of Matplotlib. It's designed specifically for statistics, making it effortless to create complex visualizations like heatmaps or violin plots that look great by default.

Plotly & Bokeh : These are the modern, interactive web-builders. They let you create charts where you can zoom, pan, and hover over data points to see details—perfect for interactive dashboards.

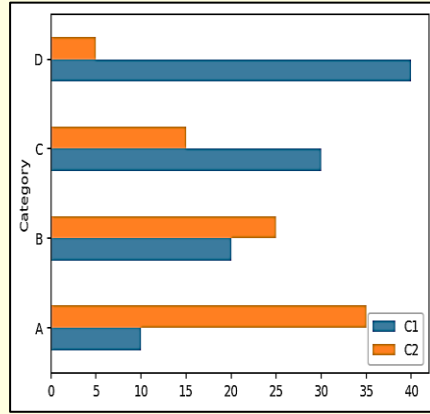
This lecture focuses only on `pandas.plot()` .

- **pandas** provides limited built-in visualization capabilities, using `.plot()` function, for quick exploratory data analysis.
- Pandas integrates with *Matplotlib*, enabling users to generate quick and effective visualizations directly from `DataFrames` without additional setup.

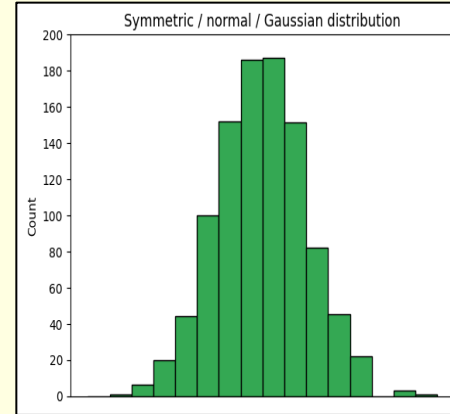
Pandas .plot() Functions



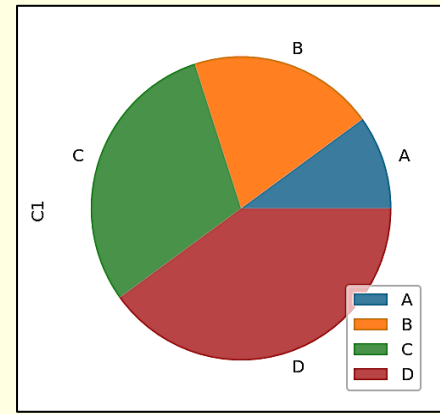
`.plot.'bar'`



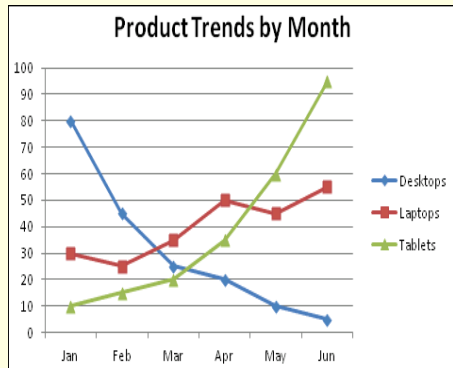
`.plot.'barh'`



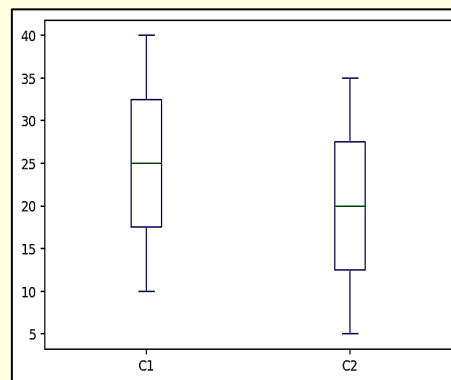
`df.plot.hist()`



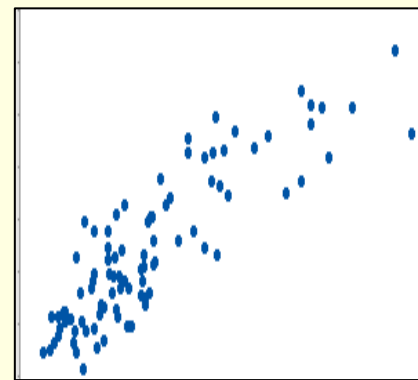
`.plot.'pie'`



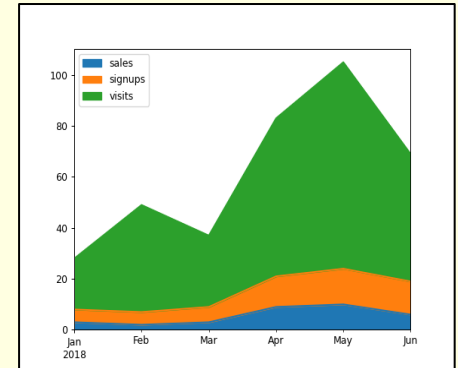
`.plot.'line'`
`.plot`



`.plot.'box'`



`.plot.'scatter'`



`df.plot.area()`

Default plot
method

Plot Types

Plot Type	Variable Types	Purpose(s)
Histogram	Numerical (single variable)	Distribution (shows how values of a single variable spread)
Pie Chart	Categorical (single variable)	Composition (show how parts make up a whole)
Box Plot	Categorical + Numerical	Comparison (compare distributions of a numerical variable across categories)
Bar Chart	Categorical + Numerical	Comparison (compare values across categories)
Stacked Bar Chart	Categorical + Numerical	Comparison and Composition (compare totals across categories and show part-to-whole)
Line Chart	Numerical vs. Numerical or Time Series	Relationship (show trends over time or between numerical variables)
Scatter Plot	Numerical vs. Numerical	Relationship (show correlation or interaction between two numerical variables)

Example

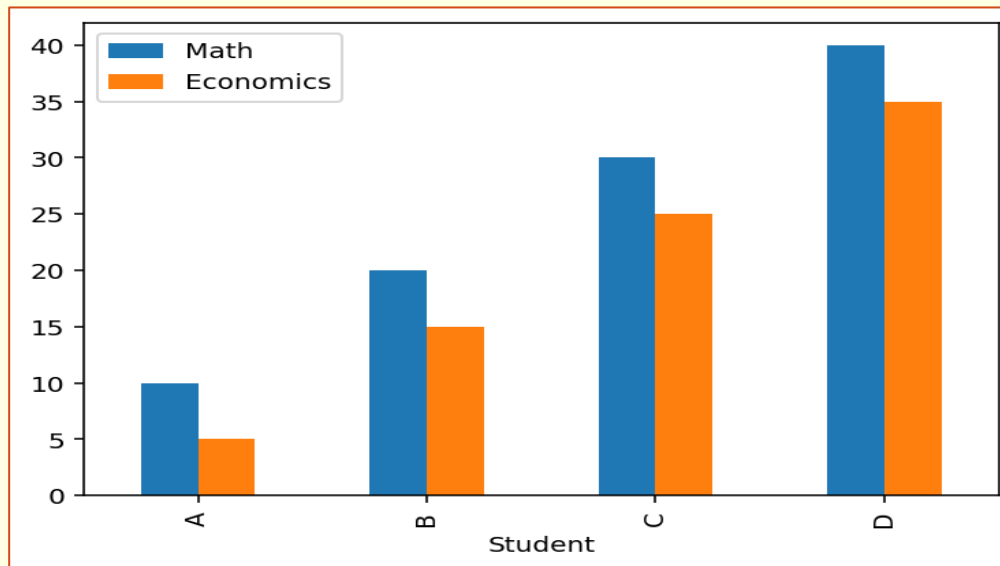
```
# Using pandas for visualization
```

```
data = { 'Student': ['A', 'B', 'C', 'D'],  
         'Math': [10, 20, 30, 40],  
         'Economics': [5, 15, 25, 35]}
```

```
df = pd.DataFrame(data)
```

```
df.plot.bar(x='Student') # Plot a bar chart
```

```
dir(df.plot) #Explore and try different kinds of plots
```



Case Study: *Titanic Dataset*



Titanic Dataset

- ***Seaborn library*** provides a collection of built-in, real-world-inspired datasets, such as tips, flights, iris, and Titanic.
- These datasets are ideal for practicing data visualization and statistical analysis.
- To get the full list of datasets available in ***Seaborn***, type
`sns.get_dataset_names()`
- The `Titanic dataset` is one of these datasets. This dataset, derived from the real historical event of the Titanic's sinking in 1912, provides information about its passengers.
- It includes details such as their sex, age, passenger class (pclass), the fare they paid, and crucially, whether they survived the disaster. This real-world data offers a compelling opportunity to analyze factors that influenced survival during this tragic event.
- The dataset contains **891 rows** and **15 columns**. We will focus only on 5 variables: 'sex', 'age', 'pclass', 'fare', 'survived'

Titanic Dataset: Load, Describe, and Prepare Data

```
import pandas as pd

import seaborn as sns # Used for loading the dataset

# --- 1. Load and Prepare Data ---

df = sns.load_dataset('titanic') # Load the Titanic dataset

print("\n--- DataFrame Info ---")

df.info()

df.shape #(891, 15)

print(df.head()) # first 5 rows

# Select certain variables and delete null and duplicate records
df_cleaned = df[['sex', 'age', 'pclass', 'survived', 'fare']]

print(df_cleaned.isnull().sum())

df_cleaned = df_cleaned.dropna(subset=['age'])

df_cleaned.shape #(714, 5)

print(df_cleaned.duplicated().sum())

df_cleaned.drop_duplicates(inplace=True)

df_cleaned.shape #(670, 5)
```

Titanic Dataset: Descriptive Analysis

--- 2. Descriptive Statistics ---

```
print("--- Descriptive Statistics for Passenger Age ---")
print(df_cleaned['age'].describe()) # a numerical variable

print("\n--- Counts for Sex ---")
print(df_cleaned['sex'].value_counts())

print("\n--- Counts for Class (pclass) ---")
print(df_cleaned['pclass'].value_counts())
#Sex and Class are Categorical Variable
```

3. Relationship between Categorical Variables using Crosstab

```
print("\n--- Survival by Sex (Row Percentages) ---")
survival_by_sex = pd.crosstab(df_cleaned['sex'],
df_cleaned['survived'], normalize='index').round(2)
print(survival_by_sex)

print("\n--- Survival by Class (Row Percentages) ---")
survival_by_pclass = pd.crosstab(df_cleaned['pclass'],
df_cleaned['survived'], normalize='index').round(2)
print(survival_by_pclass)
```

Titanic Dataset: Descriptive Analysis

```
# --- 4. Visualizations using df.plot() ---
```

```
df_cleaned['age'].plot.hist(bins=20, title='Distribution of  
Passenger Age') # Histogram shows the distribution of ages
```

```
df_cleaned.plot.scatter(x='age', y='fare', title='Passenger Age  
vs. Fare Paid') # Scatter plot: Age vs. Fare
```

```
# Bar plot for 'sex' and Class: Visualizing counts of each category
```

```
sex_count = df_cleaned['sex'].value_counts()
```

```
sex_count.plot.bar( title='Passenger Count by Sex',  
ylabel='Count')
```

```
class_count = df_cleaned['pclass'].value_counts().sort_index()
```

```
class_count.plot.bar(title='Passenger Count by Class',  
ylabel='Count')
```

Titanic Dataset: Descriptive Analysis

```
# Stacked Bar Plot: survival_by_sex and survival_by_pclass (from
crosstab data)
survival_by_pclass.plot.bar(stacked=True,
title = 'Survival Rate by Passenger Class', ylabel='Proportion',
rot=0)
survival_by_sex.plot.bar(stacked = True,
title = 'Survival Rate by Sex', ylabel = 'Proportion', rot=0)

''' show the distribution (median, quartiles, range) using a box
plot of fares paid by passengers who did survive and who did
not'''
df_cleaned.boxplot(column='fare', by='survived', sym='')
# sym='' to not display any outlier points.
```

What is Next?

What factors *best explain why* some passengers lived and others died, aboard the Titanic?



We can't answer that in this course, but you have just unlocked the tools to solve it.

Mastering Python was the critical first step.

The next step is **learning Data Science**, where you will leverage your new programming skills to drive insights in finance, policy, and research.

You are now ready to turn raw code into impactful economic solutions.

Test Your Ability

Consider the datasets below. For each one, apply the requirements below

The Annual World Happiness dataset ranks world countries by their happiness levels based on happiness scores influenced by macro variables such as GDP, social support, and life expectancy.	https://www.kaggle.com/datasets/unsdsn/world-happiness
A survey-based dataset on how COVID-19 affected people's mental health, employment, and daily life.	https://www.kaggle.com/datasets/imdevskp/corona-virus-report
Covers population statistics across different countries, including age distribution, fertility rates, and migration trends.	https://www.kaggle.com/datasets/iamsouravbanerjee/world-population-dataset

1. Download the dataset from Kaggle and read it.
2. Get the necessary information to understand the dataset structure.
3. Get descriptive statistics for the whole dataset
4. Separate categorical and numerical variables
5. Print descriptive statistics for categorical and numerical variables
6. Compute correlation matrices for numerical variables
7. Plot the convenient charts to present different variables distribution
8. Plot the convenient charts to present the relationship between different variables

Bonus → Group of 2 (3 points)

Select a Dataset: Choose one database from the suggested options or any dataset you prefer.

Apply the Analysis Requirements: Perform the required analyses as outlined above.

Prepare a Presentation: Summarize and visualize your findings clearly. Be ready to present and discuss the insights you derived from your analysis.

Evaluation Criteria: Your score will primarily be based on the quality and clarity of your live presentation, including how well you explain and interpret your results.

*Note: **You can** use AI tools like ChatGPT or Gemini, but make sure to mention and cite any help you get from them in your work.*